

ASSIGNMENT I:

Objectives

1. Manipulate data structures or algorithms in problem solving and programming.
2. Perform complexity analysis of algorithms.

Specification

Part 1.

The multiplications of 2 numbers in base 10 can be done using **simple multiplication algorithm**. The algorithm is explained below:

Step 1: Multiply each digit in the multiplicand by each digit of the multiplier, keep the partial product and carriers if there is any (e.g. $3 \times 8 = 24$, “4” is the partial product and “2” is the carrier). Print out the partial and carrier at every step.

Step 2: Add up all the properly shifted partial product and carriers.

Figure below shows an example where the multiplicand ‘52301’ is multiplied by the multiplier ‘380’ using the simple multiplication algorithm.

52301	
x 380	

00000	partial products for (=52301 x 0)
00000	carriers for (52301 x 0)
06408	partial products for (=52301 x 8)
41200	carriers for (52301 x 8)
56903	partial products for (=52301 x 3)
+ 10000	carriers for (52301 x 3)

19874380	
=====	

Implement the algorithm above in Java. Assume the length of the two numbers is the same. The algorithm must be implemented exactly as described. Insert counters at appropriate location in the program to count the primitive operations (assignment, addition, ...) executed by the simple multiplication algorithm that you wrote. Randomly generate two numbers of **n digits**, plot a graph (number of operation vs n). Compare and discuss the results obtained in the experiment and the time complexity of the theoretical analysis in Big-O.

Part 2

Besides the standard multiplication algorithm, another algorithm that can be used to calculate a multiplication is the **Karatsuba algorithm**. You are given the Karatsuba algorithm implemented in Java. **Insert counters at appropriate location in the program to count the primitive operations (assignment, addition, ...) executed by the Karatsuba algorithm and the Simple Multiplication algorithm implemented in Part 1.** You may modify the program to handle large numbers, randomly multiply two numbers of n digits, plot a graph (number of operation vs n). Compare and discuss the results obtained in the experiment and the time complexity of the theoretical analysis. Compare the time complexity of simple multiplication algorithm versus Karatsuba algorithm based on the experiment results.

This assignment is to be carried out in a group of 2. Any part submitted by a group for the evaluation must not be the same or similar to other groups' assignment. Copying from any one/place will be given ZERO. No other API, other than Java SE can be used.

Program (including the dataset) and report submission deadline: Sunday, 3 May 2026 11.59 pm is through e-learning by one of the group member.

Assignment Assessment Rubric

	Excellent (80-100%)	Good (65-79%)	Moderate (40-64%)	Poor (0-39%)	Total
Part 1a: Simple Multiplication algorithm (40%)	Algorithm is implemented in Java as described. Print out the partial products and carriers at every step (for small numbers). Algorithm is able to handle large number, e.g. n digits=10000.	Algorithm is implemented in Java as described. Print out the partial products and carriers at every step (for small numbers). Algorithm is not able to handle large number, e.g. n digits=10000.	Algorithm is implemented in Java as described. No print out of the partial products and carriers at every step (for small numbers).	Algorithm is not implemented in Java. It does not run or it is not implemented exactly as described.	
Part 1b: Implementation (27%)	Use suitable objects. Codes are efficiently implemented. Codes are properly documented.	-	Codes use unsuitable objects or codes not efficiently implemented.	Codes use unsuitable objects and codes not efficiently implemented.	

Part 2a: Counting (15%)	All counters are correctly added. Graphs are plotted.	Some counters are incorrectly added/ not added. Graphs are plotted.	Many counters are incorrectly added/ not added. Some graphs are incorrectly plotted.	No counter is added. No graphs.	
Part 2b: Analysis (18%)	Correctly specify the time complexity of the algorithms and correctly justify by analyzing the graphs obtained. Compare the two algorithms.	Correctly specify the time complexity of the algorithms and correctly justify by analyzing the graphs obtained.	Correctly specify the time complexity of the algorithms but do not analyze the graphs obtained.	No time complexity given. No analysis being carried out.	

	Does not compare the algorithms.	
--	----------------------------------	--